

SOFTWARE REQUIREMENTS SPECIFICATION

Restaurant App MVP

Frontend Only — React Native

Assignment No. 1 — Fall 2026

Student	Muhammad Abdullah
Registration No.	9667
Program	BS Software Engineering
University	Abasyn University, Islamabad Campus
Document	Software Requirements Specification (SRS)
Project Type	Restaurant App MVP — Frontend Only

Prepared for Assignment Submission

1. Introduction

The Restaurant App MVP is a frontend-only mobile application developed using React Native. The application provides a restaurant ordering and reservation experience using local and mock data. The system is designed as an MVP for demonstrating navigation, authentication, menu browsing, search, cart management, reservations, order tracking, and a manager dashboard.

This SRS defines the functional and non-functional requirements, user roles, user stories, mock-data approach, screens, navigation, and React/React Native hook usage for the application.

1.1 Purpose

The purpose of this document is to specify the requirements of the Restaurant App MVP so that the frontend implementation can be developed and evaluated consistently against the assignment requirements.

1.2 Scope

The scope is limited to the frontend of the restaurant application. The application uses local/mock data and does not require a backend server or external API. Authentication, menu information, cart state, reservations, orders, and manager information are represented on the client side for demonstration.

1.3 Definitions and Abbreviations

Term	Meaning
MVP	Minimum Viable Product
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
Frontend Only	Application functionality implemented without a backend service or external API
Mock Data	Local sample data used to demonstrate application behavior
Customer	User who browses the menu, manages a cart, places an order, and makes reservations
Manager	Restaurant-side user who views/manages mock order and reservation information

2. Overall Description

The application is a mobile restaurant frontend. Users interact with screens through React Native components and navigation. The application demonstrates two roles: Customer and Manager. Customers use the menu, search, cart, order summary, reservation, and tracking features. Managers use a dashboard to view mock restaurant activity.

The system is intentionally frontend-only. No backend implementation, external API integration, or third-party state-management library is required.

2.1 User Roles

- **Customer:** Can sign in, browse menu items, search/filter items, add items to a cart, change quantities, review an order summary, make a reservation, and track order status.
- **Manager:** Can sign in using the manager mock account and access a manager dashboard containing mock orders/reservations and management-oriented information.

2.2 User Stories

ID	User Story
US-01	As a customer, I want to log in or sign up so that I can access the restaurant app.

ID	User Story
US-02	As a customer, I want to browse menu categories and food items so that I can choose food.
US-03	As a customer, I want to search menu items so that I can find food quickly.
US-04	As a customer, I want to add items to my cart and change quantities so that I can prepare my order.
US-05	As a customer, I want to review the order summary and total so that I can confirm my order details.
US-06	As a customer, I want to make a reservation by selecting date, time, and party size.
US-07	As a customer, I want to see order progress/status so that I know the current state of my order.
US-08	As a manager, I want to view a dashboard with mock order and reservation information.

2.3 Assumptions and Constraints

- The application is a frontend-only MVP.
- Data is local/mock data; no backend or external API is required.
- Authentication is demonstrated using mock credentials.
- Application state is handled with React/React Native hooks and Context/Reducer patterns rather than an external state-management library.
- The application is intended for mobile use with React Native and Expo.

3. Functional Requirements

ID	Area	Requirement
FR-01	Authentication	The system shall provide Login and Signup modes with input validation and mock authentication.
FR-02	Role Selection	The authentication interface shall support Customer and Manager roles.
FR-03	Menu	The system shall display restaurant menu items with category, name, image, price, and an Add to Cart action.
FR-04	Search	The system shall allow the customer to search menu items by text.
FR-05	Category Filtering	The system shall allow menu items to be filtered by category.
FR-06	Scrolling	The menu shall support scrolling through available items.
FR-07	Cart	The system shall allow customers to add items to a cart, increase/decrease quantity, remove items, and clear the cart.
FR-08	Totals	The cart/order summary shall calculate subtotal and total values from item prices and quantities.
FR-09	Order Summary	The system shall display selected items, quantities, and calculated totals before order completion.
FR-10	Reservation	The system shall provide a reservation interface for date, time, and party size with validation.
FR-11	Order Tracking	The system shall display mock order status/progress for a customer order.
FR-12	Manager Dashboard	The system shall provide a manager dashboard showing mock orders and reservation information.
FR-13	Theme	The system shall provide application theme state and a light/dark theme toggle through Context.
FR-14	Authentication State	The system shall make authentication state available across relevant screens using Context.

4. Non-Functional Requirements

ID	Requirement
NFR-01	Usability: The interface should provide clear controls for menu browsing, search, cart operations, reservation, and navigation.
NFR-02	Performance: Menu rendering and derived values should use appropriate React hooks so unnecessary recalculation and rendering is avoided.
NFR-03	Maintainability: Code should be organized into components, screens, context, reducers, hooks, data, and navigation folders.
NFR-04	Reliability: User input should be validated and invalid submissions should display appropriate validation feedback.

ID	Requirement
NFR-05	Portability: The frontend should run as a React Native/Expo mobile application.
NFR-06	Security Scope: The project is an academic frontend-only MVP; authentication uses mock credentials and does not implement

5. Mock Data Requirements

The application shall use local/mock data to demonstrate functionality. The assignment does not require a backend or external API.

Data Type	Example / Purpose
Users	Customer and Manager mock accounts for authentication and role-based navigation.
Menu Items	Food item id, name, category, price, and image information.
Cart Items	Selected menu item information plus quantity.
Orders	Mock order information used for order summary/tracking and manager dashboard.
Reservations	Mock reservation information including customer/date/time/party-size fields.

6. Screens and Navigation

Screen	Purpose
Login / Signup	Authenticate the customer or manager using mock data; validate inputs.
Menu	Display categories and menu cards; search/filter and add items to cart.
Cart	Display selected items, quantities, item removal, and totals.
Order Summary	Review order items, quantities, and calculated totals.
Reservation	Enter/select reservation date, time, and party size with validation.
Order Tracking	Display mock order status/progress.
Manager Dashboard	Display mock orders/reservations and manager-facing information.

6.1 Navigation Flow

- Application starts at Login/Signup.
- Successful Customer authentication navigates to the Menu.
- Successful Manager authentication navigates to the Manager Dashboard.
- Customer flow includes Menu → Cart → Order Summary → Order Tracking.
- Customer can access Reservation from the application navigation.
- Authentication state is shared using AuthContext.

7. React / React Native Hooks Requirements

Hook	Planned Use in Application
useState	Form fields, selected category, search text, loading/submission state, theme state, and other local UI state.
useEffect	Side effects such as loading/persisting local data or reacting to relevant state changes.
useRef	Reference to a component/input or other mutable value where a render should not be triggered.
useContext	Access authentication and theme state throughout the application.
useReducer	Manage cart state and cart actions such as add, remove, increase/decrease quantity, and clear.
useMemo	Derive filtered menu data and calculated values where appropriate without unnecessary recalculation.

Hook	Planned Use in Application
useCallback	Memoize callback functions passed to child components where appropriate for performance.
Custom Hooks	Reusable application logic such as a cart, persistence, or other shared hook abstraction.

8. Feature-Specific Requirements

8.1 Q3 — Login / Signup

- Provide Login and Signup modes.
- Validate email and password input on submit.
- Signup validates full name, email, password, and confirmation password.
- Use mock credentials/data for authentication.
- Show a loading state/delay during authentication to demonstrate asynchronous UI behavior.
- Navigate Customer users to Menu and Manager users to Dashboard.
- Provide user-visible validation/error feedback.
- Required screenshots include a validation-error state and a successful-login state.

8.2 Q4 — Menu

- Display menu categories and food cards.
- Each food card includes image, name, category, price, and Add to Cart.
- Use local/mock menu data.
- Support category filtering and menu browsing.

8.3 Q5 — Search and Scroll

- Provide menu search using a text input.
- Display matching menu items as the search text changes.
- Support vertical scrolling through menu items and horizontal category browsing where applicable.

8.4 Q6 — Authentication and Theme Context

- Create AuthContext for authentication state and login/logout actions.
- Create ThemeContext for theme state and theme toggling.
- Wrap the application with the relevant providers.
- Use useContext in screens/components that need authentication or theme information.

8.5 Q7 — Cart

- Use useReducer for cart state.
- Add an item to the cart; adding an existing item increases its quantity.
- Increase and decrease quantities.
- Remove items and clear the cart.
- Calculate and display subtotal/total.

8.6 Q8 — Order Summary and Performance

- Display the selected order items and quantities.
- Display calculated totals.

- Use appropriate memoization/performance hooks where required by the assignment, including useMemo and useCallback.
- Use local/mock order data; no backend is required.

8.7 Q9 — Reservation

- Provide a reservation form for date, time, and party size.
- Validate required reservation fields.
- Display a confirmation/result using local/mock data.
- The feature remains frontend-only.

8.8 Q10 — Order Tracking and Manager Dashboard

- Display mock order status/progress for customers.
- Provide a manager dashboard with mock orders and reservation information.
- The dashboard is for frontend demonstration and does not require a backend service.

9. UI / UX Requirements

- Use clear labels and controls for authentication, search, categories, cart actions, reservations, and navigation.
- Provide readable feedback for validation and successful actions.
- Keep layouts appropriate for a mobile screen.
- Use scrolling for content that exceeds the available screen area.
- Provide consistent navigation and visual treatment across screens.
- Support a light/dark theme through the theme context.

10. Project Structure

The application source is organized to match the assignment structure:

RestaurantApp/

■■■■ src/

■ ■■■■ components/

■ ■■■■ screens/

■ ■■■■ context/

■ ■■■■ reducers/

■ ■■■■ hooks/

■ ■■■■ data/

■ ■■■■ navigation/

■■■■ A1/

■ ■■■■ SRS.pdf

■ ■■■■ UML/

■■■■ App.js

■■■■ README.md

■■■■ package.json

11. Testing / Acceptance Criteria

ID	Acceptance Check
TC-01	Invalid login input displays validation feedback.
TC-02	Valid Customer mock credentials navigate to Menu.
TC-03	Valid Manager mock credentials navigate to Dashboard.
TC-04	Menu displays item information and categories.
TC-05	Search/filter changes the displayed menu items.
TC-06	Customer can add an item to the cart.
TC-07	Customer can increase, decrease, remove, and clear cart items.
TC-08	Order summary displays quantities and calculated totals.
TC-09	Reservation form validates required fields.
TC-10	Order tracking displays a mock order status/progress.
TC-11	Manager dashboard displays mock order/reservation information.
TC-12	Theme/authentication state is accessible through Context.

12. Submission Requirements

- GitHub repository name: **restaurant-app-mvp**.
- Include the source structure under `src` with components, screens, context, reducers, hooks, data, and navigation.
- Include **A1/SRS.pdf**.
- Include the five UML diagrams under **A1/UML**.
- Use a meaningful Git commit for each assignment question/major feature.
- README shall include installation/start instructions, mock credentials, hook usage table, screenshots, and a video/demo link.
- The project shall remain frontend-only; no backend or external API is required.

13. Limitations

This SRS describes a frontend-only academic MVP. It does not specify production backend services, real payment processing, real-time order synchronization, server-side authentication, database design, or external API integration. Such functionality is outside the stated frontend-only scope.

14. Conclusion

The Restaurant App MVP provides a defined frontend scope covering authentication, menu browsing, search, cart, order summary, reservation, order tracking, theme/authentication context, and manager dashboard functionality. The SRS provides the requirements and organization needed to implement and evaluate the application as a React Native frontend using local/mock data and the required React hooks.